

Recursos ISIS-2211

Angular

¿Por qué existe un framework?

Antes de entender qué es Angular, es más útil entender **qué problema vino a resolver**.

Cuando una aplicación web es pequeña, una página con tres botones, JavaScript vanilla es suficiente. El problema aparece cuando la aplicación crece.

El modelo mental de JavaScript vanilla

En JavaScript puro, el HTML que el usuario ve vive en el DOM. El estado de la aplicación (datos que el código conoce) vive en variables. **No hay ningún mecanismo automático que los mantenga sincronizados.**

Cada vez que el estado cambia, el desarrollador debe recordar qué parte del DOM reflejar ese cambio y escribir el código imperativo para hacerlo. Esto funciona, pero no escala.

 **Vanilla JS, código imperativo**

```
// El estado vive aquí
let contador = 0;

// El DOM vive en otro lugar
const btn = document.querySelector('#btn');
const span = document.querySelector('#valor');

// El desarrollador sincroniza manualmente
btn.addEventListener('click', () => {
  contador++;
  span.textContent = contador; // ← obligatorio: sin esta línea el valor
  // del contador cambia pero el usuario sigue viendo el anterior
});
```

Con 3 variables y 2 elementos funciona. Con 50 variables y 200 elementos, olvidar una sincronización produce bugs silenciosos.

Angular, declarativo

```
// El estado vive en la clase
contador = 0;

// El template declara la relación
// <span>{{ contador }}</span>
// <button (click)="contador++">

// Angular se encarga de la sincronización.
// El desarrollador solo describe
// qué debe mostrarse, no cómo hacerlo.
```

La cantidad de elementos en pantalla puede crecer sin agregar complejidad a la sincronización.

Actividad interactiva: Comparación imperativo vs declarativo

Aquí está el contraste en acción. Nota cómo en el lado vanilla el código **sincroniza manualmente**, mientras que en Angular el framework **sincroniza automáticamente**.



Imperativo vs Declarativo



Vanilla JS (Imperativo)

```
<!-- HTML -->
<button id="btn">Incrementar</button>
<p>Resultado: <span id="val">0</span>
</p>

// TypeScript
const btn =
document.querySelector('#btn');
const val =
document.querySelector('#val');
let contador = 0;

btn.addEventListener('click', () => {
  contador++;
  val.textContent = contador;
});
```



Angular (Declarativo)

```
<!-- Template -->
<button (click)="contador++">
  Incrementar
</button>

<p>Resultado: {{ contador }}</p>

// TypeScript
contador = 0;
```

Incrementar

Variable contador0Lo que ve el usuario0

Incrementar

Estado contador0Lo que ve el usuario0

Aquí no hay interruptor: no existe esa línea.

<https://recursos-isis-2211.virtual.uniandes.edu.co/temas/frontend/angular/>

Página 3 de 25

✓ Ejecutar la línea de sincronización

⚠ El HTML solo pone id; la conexión se arma después, en el TypeScript. Apaga la línea y sigue haciendo clic: la variable cambia, la pantalla no. Mantenerlas iguales es trabajo tuyo.

✓ La conexión se declara en el propio HTML: (click) dice qué pasa al hacer clic y {{ }} dice qué mostrar. No queda ninguna línea de sincronización que puedas olvidar.



Concepto: los dos botones dicen "Incrementar" y hacen lo mismo. La diferencia está en **dónde vive la conexión**, lo resaltado en azul. En vanilla el HTML es inerte: solo aporta id para que el TypeScript encuentre los elementos y los conecte a mano. En Angular la conexión se declara en el mismo HTML, y por eso no queda una línea de sincronización que olvidar. Cuando se olvida, el error es silencioso: nada se rompe, solo se muestra un valor viejo.

Qué pasa cuando una app crece sin framework

Complejidad que aparece

- Estado disperso en múltiples variables globales
- Código duplicado para actualizar vistas similares
- Difícil reutilizar un "componente" de UI
- Sin convención de estructura, cada equipo inventa la suya
- Las pruebas deben simular el DOM completo
- Gestionar dependencias entre módulos se vuelve manual

Lo que un framework aporta

- Estado encapsulado por componente o servicio
- Sincronización automática vista ↔ estado

- Unidad de reutilización formal: el componente
- Estructura de proyecto predefinida y compartida
- Arquitectura que facilita las pruebas unitarias
- Sistema de inyección de dependencias integrado

Nota

Angular no es el único framework que resuelve este problema, React, Vue y otros más, lo abordan también, con filosofías distintas.

El cambio conceptual clave: de imperativo a declarativo

El salto mental más importante al adoptar Angular no es aprender su sintaxis, es cambiar la forma de pensar:

Consejo

En JavaScript vanilla se dice **cómo** actualizar la UI (busca el elemento, cambia su texto, agrega esta clase). En Angular se declara **qué** debe mostrarse en función del estado, y el framework se encarga del cómo.

Esto es posible porque Angular **compila** los templates: convierte el HTML con sintaxis Angular en código JavaScript optimizado que sabe exactamente qué partes del DOM deben actualizarse cuando cada pieza del estado cambia.



El modelo de componentes

La unidad fundamental de Angular, el *building block*, es el **componente**. Todo lo que el usuario ve en pantalla está construido con componentes. Entender qué es un componente, qué contiene y cómo se relacionan entre sí es la base de todo lo demás.

Nota

¿Qué convierte una clase TypeScript en componente de angular? Un decorado o anotación llamado **@Component**. Este decorador tiene varios atributos que permiten que el desarrollor indique características del componente y en particular que lo relacione con su vista. La información de la anotación son metadatos que le permiten al framework al compilarse, reflejar el comportamiento correcto, que vamos aver enseguida, y evitar que el desarrollador tenga que hacerlo a la mano.

¿Qué es un componente?

Un componente es una **pieza de UI autocontenida** que encapsula tres cosas:

Template (.html), Estructura

El HTML que describe qué se renderiza. Puede contener bindings (`{{ }}`), directivas (`@if`, `@for`) y eventos (`((click))`).

Clase (.ts), Comportamiento

TypeScript que define el estado del componente (propiedades) y su lógica (métodos). El template solo accede a lo que la clase expone.

Estilos (.css), Apariencia

CSS **encapsulado**: los estilos de un componente no afectan a otros componentes por defecto. Cada componente tiene su propio ámbito de estilos. Si no se usa este atributo los estilos de

con este atributo, los estilos de este componente corresponden a los globales de la aplicación.

📘 Nota

Los tres archivos se declaran juntos con el decorador **@Component**, como atributos, Angular los trata como una unidad.

```
@Component({
  selector:      'app-album-card', // etiqueta HTML personalizada
  templateUrl:   './album-card.component.html',
  styleUrls:     './album-card.component.css',
  standalone:    true,             // Angular 17+: sin NgModule
})
export class AlbumCardComponent {
  titulo: string = 'Thriller';
  anio:    number = 1982;

  reproducir() {
    console.log(`Reproduciendo ${this.titulo}`);
  }
}
```

📘 Nota

Note que el componente no sabe dónde en el layout de la aplicación lo van a desplegar. Esa responsabilidad la tiene quien lo *llame*.

Declarativo en la práctica: binding y control flow

El template Angular extiende el HTML con sintaxis para reflejar el estado declarativamente:

Sintaxis	Nombre	Qué hace
<code>{{ expresion }}</code>	Interpolación	Muestra el valor de una expresión como texto
<code>[propiedad]="valor"</code>	Property binding	Pasa un valor a una propiedad del elemento o componente hijo
<code>(evento)="handler()"</code>	Event binding	Reacciona a eventos del DOM o del componente hijo
<code>@if (condicion) { ... }</code>	Control flow	Renderiza el bloque solo si la condición es verdadera
<code>@for (item of lista; track item) { ... }</code>	Control flow	Repite el bloque por cada elemento de la lista. track es obligatorio

Actividad interactiva: Practica binding y control flow

Aquí puedes practicar la sintaxis de los templates. Hay dos paneles: uno muestra cómo se ve el código ejecutado, el otro muestra ejemplos de cada tipo de binding.



Binding y Control Flow de Angular

Los cinco ejemplos son de un mismo componente. Cambia el estado en la columna izquierda y observa cómo el template lo refleja.

{{ }} Interpolación

Muestra el valor de una propiedad como texto

CLASE (.TS)

TEMPLATE (.HTML)

RESULTADO

```
nombre = 'Juan';
```

```
<p>Hola {{ nombre }}</p>
```

Hola Juan

Juan

[propiedad] Property Binding

Enlaza una propiedad del elemento a un valor de la clase

CLASE (.TS)

```
bloqueado = true;
```

Cambiar bloqueado

TEMPLATE (.HTML)

```
<input  
  [disabled]="bloqueado"
```

RESULTADO

Escribe aquí

(evento) Event Binding

Ejecuta un método de la clase cuando ocurre un evento

CLASE (.TS)

```
clicks = 0;  
  
contar() {  
  this.clicks++;  
}
```

El botón vive en el template

TEMPLATE (.HTML)

```
<button (click)="contar()"  
  Sumar  
</button>  
  
<p>Clicks: {{ clicks }}</p>
```

RESULTADO

Sumar

Clicks: 0

@if Control Flow

Renderiza el bloque solo si la condición se cumple. No lo oculta: lo saca del DOM.

CLASE (.TS)

TEMPLATE (.HTML)

RESULTADO

```
mostrar = false;
```

☐ mostrar

```
@if (mostrar) {
  <p>iAparecí!</p>
}
```

QUÉ QUEDA EN EL DOM

```
@if (mostrar)
```

```
[hidden]="!mostrar"
```

```
<div>
</div>
```

```
<div>
  <p hidden>iAparecí!</p>
</div>
```

El nodo no existe mientras la condición sea falsa.

El nodo existe siempre; solo deja de dibujarse.

En pantalla se ven igual. La diferencia está en el marcado, y por eso importa: lo que @if saca del DOM deja de ocupar memoria y de responder a eventos.

@for Control Flow

Repite el bloque por cada elemento de una lista

CLASE (.TS)

```
items = [
  'Item 1',
  'Item 2',
];
```

+ Agregar

TEMPLATE (.HTML)

```
@for (item of items;
      track item) {
  <li>{{ item }}</li>
}
```

RESULTADO

Item 1

Item 2



track es obligatorio en @for: le dice a Angular cómo identificar cada elemento para no rehacer la lista entera cuando cambia.

El árbol de componentes

Una aplicación Angular es un árbol de componentes. El componente raíz (AppComponent) contiene a todos los demás. Cada componente puede contener otros componentes, formando una jerarquía.

Nota

La idea de árbol de componentes no es exclusiva de Angular, es el modelo mental de todo framework moderno (React, Vue, Svelte). Una vez internalizado en Angular, los otros frameworks resultan familiares.

```
— AppComponent (raíz)
  |— NavbarComponent
  |— RouterOutlet ← el router inserta el componente activo aquí
  |   |— AlbumListComponent
  |   |   |— AlbumCardComponent (se repite con @for)
  |   |   |— AlbumDetailComponent
  |— FooterComponent
```

Componer: usar un componente dentro de otro

Componer es poner el selector de un componente dentro del template de otro, como si fuera una etiqueta HTML propia. Ese acto es el que crea la relación padre → hijo y, al repetirse, forma el árbol.

```
<!-- album-list.component.html - el padre -->
<h2>Mis álbumes</h2>

<app-album-card />    <!-- ← esto es composición -->
```

El hijo no sabe nada de esto: AlbumCardComponent ignora quién lo usa y dónde aparece en la pantalla. Toda la responsabilidad de ubicarlo es del padre, que es quien lo *llama*.

⚠ Componer no es lo mismo que importar

Es la confusión más común al empezar, sobre todo viniendo de React. Para que `<app-album-card />` funcione hacen falta **dos pasos distintos**:

Paso	Dónde	Qué hace
<code>import { AlbumCardComponent } from '...'</code>	Arriba del archivo <code>.ts</code>	Trae la clase al archivo. Es TypeScript puro, nada de Angular.
<code>imports: [AlbumCardComponent]</code>	Dentro del <code>@Component</code>	Le dice al compilador de Angular que el selector de ese componente puede resolverse en este template.
<code><app-album-card /></code>	En el <code>.html</code>	Esto es la composición. Instancia el hijo dentro del padre.

En Angular el template es un archivo aparte que no ve las variables del `.ts`: por eso el decorador necesita el array `imports` como puente.

Dicho corto: `imports` es requerido para componer; la composición ocurre en el HTML.

```
import { Component } from '@angular/core';
import { AlbumCardComponent } from './album-card.component'; // 1.
import de TypeScript

@Component({
  selector: 'app-album-list',
  imports: [AlbumCardComponent], // 2. permiso para usar <app-
album-card> en el template
  templateUrl: './album-list.component.html',
})
export class AlbumListComponent { }
```

📌 Nota

Si falta el paso 2, Angular no reconoce la etiqueta y falla la compilación. Si sobra (se importa un componente que el template nunca usa), Angular avisa que el import no se está utilizando.

Comunicación entre componentes

Los componentes se pueden comunicar de distintas formas. Una de ellas es a través de un contrato formal que el hijo declara y el padre completa. La pregunta que decide cuál mecanismo usar no es "¿en qué dirección van los datos?", sino **¿quién tiene permiso de escribir este valor?**

Mecanismo	Quién escribe el valor	Cómo lo ata el padre	Qué es en el hijo
<code>input()</code>	Solo el padre. El hijo lo lee	<code>[titulo]="album"</code>	Un <code>InputSignal</code> , de solo lectura : no tiene <code>set()</code>

llamándolo:
`titulo()`.

<code>output()</code>	El padre, cuando el hijo se lo pide emitiendo un evento	<code>(reproducir)="poner(\$event)"</code>	Un <code>OutputEmitterRef</code> (no es un signal); se usa con <code>.emit(valor)</code>
<code>model()</code>	El hijo, y el padre se entera	<code>[(sonando)]="sonando"</code>	Un <code>ModelSignal</code> , que sí se puede escribir
Servicio compartido	Cualquiera que lo inyecte	No se ata en el template: <code>inject(MiServicio)</code>	Un tercero que guarda el estado, útil entre componentes no relacionados

Con `model()` el padre no tiene que declarar nada más: Angular le crea al hijo un evento de salida con el nombre del input más `Change` (para sonando, un `sonandoChange`), y eso es lo que hace funcionar la sintaxis `[( )]`.

Actividad interactiva: Comunicación entre componentes

Una tarjeta quiere ponerse a tocar una canción. Pruebe `input()` y `output()` —los dos mecanismos que compiten por resolver este problema— y observe qué pasa con la *otra* tarjeta: ahí está la diferencia entre escribir el valor ella misma y pedírselo al padre. `model()` y el servicio compartido, de la tabla de arriba, resuelven un problema distinto: uno en el que el valor sí le pertenece al hijo.



Comunicación entre componentes

La tarjeta quiere tocar una canción. ¿Cómo lo consigue?

- `output()`: le avisa al padre ○ `input()`: se lo cambia ella misma

Clase del padre · `album-list.component.ts`

```
export class AlbumListComponent {
  sonando = signal<string | null>(null);
  poner(t: string) {
    this.sonando.set(t); }
}

<!-- template -->
<app-album-card
  [titulo]=" 'Thriller' "
  [sonando]="sonando() === 'Thriller' "
  (reproducir)="poner($event)" />
<app-album-card [titulo]=" 'The Wall' " ...
/>
```

Clase del hijo · `album-card.component.ts`

```
import { input, output } from
 '@angular/core';

export class AlbumCardComponent {
  titulo      = input.required<string>();
  sonando     = input(false);
  reproducir  = output<string>();

  alHacerClic() {
    this.reproducir.emit(this.titulo());
  }
}
```

PADRE AlbumListComponent *aquí vive el estado*

sonando() **null**

↓ [titulo] [sonando]

↑ (reproducir)

HIJO AlbumCardComponent

titulo() **'Thriller'**

sonando() **false**

HIJO AlbumCardComponent

titulo() **'The Wall'**

sonando() **false**

Thriller

▶ Reproducir

The Wall

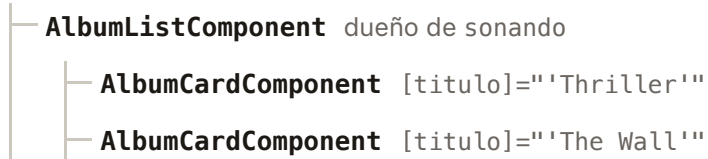
▶ Reproducir

Haz clic en una tarjeta y luego en la otra.

 Reiniciar

Árbol de componentes

AppComponent



Es el mismo componente hijo dos veces, con un `input()` distinto cada uno. Los datos bajan por los corchetes `[ ]` y los eventos suben por los paréntesis `( )`.



Concepto: la pregunta no es "¿input o output?", sino **¿quién tiene permiso de escribir este valor?** Con `input()`, solo el padre; por eso el intento de la tarjeta ni compila. Con `output()`, el padre puede escribirlo cuando el hijo se lo pide, y por eso sí logra coordinar a las dos tarjetas. El estado que *coordina* a varios hijos tiene que vivir en el padre.

Nota

En tutoriales y proyectos anteriores a Angular 19 esto mismo se escribe con **decoradores**: `@Input()` `titulo = ''` y `@Output()` `reproducir = new EventEmitter<string>()`. Hacen lo mismo, siguen soportados, y el equipo de Angular recomienda las funciones (`input()`, `output()`) para código nuevo. La diferencia que importa para entender el concepto: con `@Input()` la propiedad es una variable común y corriente, así que el hijo *puede* escribirla y el error —que el padre nunca se entere— es silencioso. Con `input()` no compila.

Veremos más sobre cómo comunicar componentes más adelante.

Arquitectura de Angular

Angular es un framework **prescriptivo** (*opinionated*, en inglés). Esto significa que el framework ya tomó decisiones de diseño por el equipo: cómo estructurar el proyecto, dónde vive la lógica, qué herramientas usar para routing, HTTP y formularios. No ofrece libertad total, pero reduce la fricción inicial y facilita el trabajo en equipo, porque todos siguen las mismas convenciones. Conocer sus piezas principales permite tomar decisiones de diseño informadas.

Las piezas del framework

Pieza	Responsabilidad	Analogía
Componente	Unidad de UI + comportamiento	Un widget con estado propio
Servicio	Lógica reutilizable, acceso a datos	La capa de negocio del frontend
Router	Navegar entre vistas según la URL	El controlador de flujo de la app

HttpClient	Comunicación con APIs REST	El cliente HTTP del backend
Formularios reactivos	Captura y validación de input del usuario	Estado del formulario como objeto TypeScript
DI (Inyección de Dependencias)	Proveer instancias de servicios a quien los necesite	El contenedor de <i>Spring</i> , en el frontend

Standalone components (Angular 17+)

Hasta Angular 15 cada componente debía pertenecer a un modulo, una clase TS decorada con **NgModule**, la clase declaraba que componentes, directivas y servicios y declaraba sus dependencias a otros módulos. A partir de Angular 17 los componentes pueden ser **standalone**: se autodescriben, declaran sus propias importaciones y no necesitan NgModule.

Antes, con NgModule

```
@NgModule({
  declarations: [AlbumListComponent],
  imports:      [CommonModule, HttpClientModule],
  exports:      [AlbumListComponent],
})
export class AlbumModule {}
```

Archivos adicionales obligatorios aunque el módulo hiciera poco.

Ahora, standalone

```
@Component({
  standalone: true,
  imports: [HttpClientModule],
  template: `...`,
})
export class AlbumListComponent {}
```

El componente declara sus dependencias directamente. Sin módulo intermediario.

El ciclo de vida de la compilación

Angular no ejecuta los templates en tiempo de ejecución. Los templates son **compilados** por el Angular Compiler durante el build a JavaScript optimizado. Esto tiene dos consecuencias importantes:

- Los **errores de template** se detectan en tiempo de compilación, no en el browser.
- El código resultante es más pequeño y rápido porque Angular sabe exactamente qué puede cambiar.

Señales (Signals), el nuevo modelo de reactividad

Angular 17 introdujo las **signals** como el mecanismo de reactividad de primera clase, complementando y gradualmente reemplazando a Zone.js.

- 1 **Angular 1–15 · Zone.js**, Angular detecta cambios “envolviendo” las APIs del browser (setTimeout, eventos, HTTP). Cuando cualquiera de ellas se ejecuta, Angular revisa todo el árbol de componentes. Funciona, pero es poco preciso.
- 2 **Angular 16 · Signals (preview)**, Se introduce `signal()`, `computed()` y `effect()`. Un signal notifica exactamente qué cambió, Angular actualiza solo el componente que depende de ese valor.

- 3 **Angular 17+ · Signals estables**, Signals son la forma recomendada para manejar estado local en componentes. Más predecible, mejor rendimiento, sin necesidad de Zone.js en el futuro.
- 4 **Angular 19 · La comunicación entre componentes también pasa a signals**, `input()`, `output()` y `model()` se vuelven estables y reemplazan a los decoradores `@Input()` y `@Output()` en código nuevo.

Precaución

En este curso trabajaremos con Angular 20 (la versión que instala el taller) usando standalone components y signals. Si encuentra tutoriales más viejos que usan NgModules o la sintaxis `*ngIf` / `*ngFor`, son válidos técnicamente pero corresponden a Angular 14 o anterior. La sintaxis nueva (`@if` / `@for`) es más limpia y es la que se mantiene a futuro.

Angular en el stack completo

Ya conoce la arquitectura del backend: tres capas (API REST, Lógica de Negocio, Persistencia) construidas en Java y Spring Boot. Angular es la **capa cliente** que vive en el browser del usuario y se comunica con ese backend a través de HTTP.

El stack completo visualizado

EL STACK COMPLETO

La frontera crítica: el contrato HTTP

Angular y Spring Boot son independientes: hablan a través de HTTP con JSON. El contrato entre los dos lados, las URLs, los verbos, los DTOs, es lo que estudiaste en los artefactos del backend. Angular consume exactamente esos endpoints.

Consejo

Un cambio en el DTO de respuesta del backend *rompe* el frontend si no se actualiza el modelo TypeScript correspondiente. El contrato HTTP es la interfaz compartida entre equipos. Mantenerlo estable (o versionarlo) es fundamental.

Mapeo conceptual: backend ↔ frontend

Concepto backend	Equivalente Angular	Qué conecta
Controller / @RestController	Componente Angular	Recibe la interacción del usuario, delega al servicio
Service / @Service	Servicio Angular (@Injectable)	Encapsula la lógica de llamadas HTTP y reglas del cliente
DTO (Data Transfer Object)	Interface o clase TypeScript	Define la forma del JSON que viaja entre capas
Repository	HttpClient en el servicio Angular	Accede a la fuente de datos, en este caso, la API REST
Validaciones (@NotNull, etc.)	Validators en formularios reactivos	Reglas de validación, en el cliente y en el servidor

Consejo

La validación del cliente (Angular) **no reemplaza** la validación del servidor (Spring Boot). El frontend valida para dar feedback rápido al usuario; el backend valida para garantizar la integridad del sistema. Siempre ambas.

Responsabilidades del frontend que no tiene el backend

Angular tiene responsabilidades que no existen en el servidor:

Estado de la UI

- ¿Está el menú desplegado?
- ¿Qué pestaña está activa?
- ¿Hay una operación en curso (loading)?
- ¿Qué mensaje de error mostrar?

Experiencia del usuario

- Navegación sin recargar la página
- Feedback inmediato en formularios
- Actualización parcial de la vista
- Caché de datos ya cargados

Nota

El backend que construyeron en las capas anteriores del curso expone una API REST completa. En los próximos artefactos vamos a consumirla desde Angular: listar álbumes, ver el detalle de uno, crear uno nuevo y eliminarlo, todo el CRUD, desde el browser.

Autoevaluación

¿Ya tiene claro por qué Angular existe y cómo se arma un componente? Compruébelo con 6 preguntas antes de llegar al primer taller. No afecta la nota, es su radar personal. Si se equivoca, la explicación le dice por qué esa opción no es, sin revelarle cuál sí es, para que lo razone usted mismo.

1. Un compañero escribió un componente que actualiza el contador con `this.contador++`, pero la pantalla nunca cambia. ¿Cuál es la causa más probable?

- ☐ Angular no detecta el cambio porque el template no está enlazado a esa propiedad con interpolación o binding
- ☐ TypeScript no permite el operador `++` dentro de una clase
- ☐ El navegador no soporta la sintaxis de Angular
- ☐ El componente necesita ser standalone para que el template se actualice

2. ¿Qué convierte una clase TypeScript cualquiera en un componente de Angular?

- ☐ Que el archivo termine en .component.ts
- ☐ El decorador @Component con sus metadatos
- ☐ Que la clase tenga un método ngOnInit()
- ☐ Que se importe en el archivo main.ts

3. ¿Cuál de las siguientes es una ventaja real que aporta un framework como Angular frente a escribir todo en JavaScript vanilla?

- ☐ Sincroniza automáticamente la vista con el estado, sin que el desarrollador escriba el código imperativo para actualizar el DOM
- ☐ Elimina la necesidad de escribir HTML
- ☐ Hace que el código se ejecute más rápido que JavaScript puro en el navegador
- ☐ Permite que la aplicación funcione sin conexión a internet

4. El material describe Angular como un framework prescriptivo (opinionated). ¿Qué ventaja trae esto para un equipo de desarrollo?

- ☐ Los desarrolladores tienen total libertad para elegir cualquier herramienta de routing, HTTP o formularios
- ☐ El framework ya tomó decisiones de diseño (estructura, routing, HTTP, formularios), lo que reduce la fricción y facilita que todo el equipo siga las mismas convenciones
- ☐ El código se ejecuta directamente en el servidor sin necesidad de un navegador
- ☐ No es necesario aprender TypeScript para trabajar con Angular

5. ¿Por qué Angular 17+ ya no exige NgModule para declarar un componente?

- ☐ Porque NgModule fue eliminado del framework por completo
- ☐ Porque los componentes standalone declaran sus propias importaciones
- ☐ Porque TypeScript ahora resuelve automáticamente las dependencias
- ☐ Porque los componentes standalone no pueden tener dependencias

6. ¿Por qué un signal es más preciso que Zone.js para detectar cambios?

- ☐ Porque un signal notifica exactamente qué cambió, y Angular solo actualiza lo que depende de ese valor
- ☐ Porque Zone.js fue eliminado de Angular
- ☐ Porque los signals se ejecutan en un hilo separado del navegador
- ☐ Porque los signals no necesitan estar dentro de una clase

Comprobar